

1. (0,25p) Descrierea modelului ales și a obiectivelor aplicației - obligatoriu

Proiectul nostru descrie gestionarea unui lanț de săli de fitness. Modelul este reprezentat de către o bază de date relațională, care gestionează diferite săli, clienții salilor, angajații salilor, evenimentele ce au loc în facilități, aparatele disponibile în locații, produsele pe care clienții le pot cumpăra.

Baza de date cuprinde mai multe centre de fitness care sunt amplasate pe teritoriul României, în diverse orașe. Fiecare oraș aparține de una dintre cele 3 regiuni mari ale României: Muntenia, Transilvania și Moldova. Fiecare sală de fitness cuprinde diferite departamente, în care lucrează angajați cu diferite funcții (instructor fitness, femeie de servici etc.). O sală poate să susțină clase de fitness, specifice pentru fiecare disciplină în parte din zona fitness-ului (aerobic, body building, crossfit etc.). De asemenea, o sală poate organiza evenimente unice, la care accesul se face pe baza unui bilet nominal, separat de abonamentele lunare pe care un client trebuie să le achiziționeze. Un abonament poate oferi acces la una sau mai multe săli, în funcție de planul ales. Fiecare sală deține diferite aparate, având diferite specificații. Un client poate achiziționa, pe lângă cele menționate mai sus, și produse cu specific fitness din fiecare locație (suplimente, shake-uri proteice etc.).

3. (0,25p) Descrierea modului de distribuire (numărul de server-e de baze de date din rețea) - obligatoriu

Sistemul de baze de date distribuit pentru gestionarea unui lanț de săli de fitness va fi distribuit în 4 server-e de baze de date după cum urmează:

- Server-ul 1 (S1): este un server local de baze de date, ce include regiunea Transilvania
- Server-ul 2 (S2): este un server local de baze de date, ce include regiunea Muntenia
- Server-ul 3 (S3): este un server local de baze de date, ce include regiunea Moldova
- Server-ul 4 (S_GLOBAL): este un server global de baze de date, ce gestionează întregul sistem distribuit.

Am ales 3 server-e locale (per regiune geografica) si un server global din urmatoarele considerente:

- Localizarea naturala a datelor: Ideea din spatele bazei de date are in centru salile de fitness (tabela SALI_DE_FITNESS). Ele sunt localizate in orase (tabela ORAS), iar fiecare oras apartine de o regiune. Partitia geografica face mult mai usoara distribuirea server-elor. Fiecare server local va tine datele care sunt cele mai accesate in salile pozitionate in fiecare regiune, astfel reducand latenta de retea.
- Autonomie locala: Fiecare server local poate procesa o parte din cereri pentru regiunea de care apartine fara sa acceseze alte server-e.
- Coordonare globala: Serverul global este configurat sa ofere vizualizari, aliasuri, legaturi de baze de date, permitand aplicatiilor sa creeze cereri pe toate datele din toate server-ele noastre.
- Scalabilitate: Daca lantul de sali de fitness se extinde si in alte tari inafara de Romania, atunci se vor adauga noi server-e locale ce corespund noilor regiuni adaugate. Astfel, arhitectura deja create ramane intacta.
- Distribuirea echilibrata a datelor: Cu cele 3 regiuni mentionate, volumul de date este aproximativ egal distribuit intre server-ele locale.

4. (3p) Argumentarea deciziei de fragmentare a relațiilor

a. (1p) Fragmentare orizontală primară

Fragmentarea orizontala primara foloseste predicate definite direct pe relatia care este fragmentata. In cele ce urmeaza, vom aplica algoritmul COMP_MIN, urmat de algoritmul PRIM_ORIZ pentru a construi un set complet si minimal de predicate simple.

i. Aplicarea algoritmului de fragmentare orizontală primară (exemplificarea pașilor algoritmului pe baza unor date ipotetice)

Relatia target va fi ORAS. Tabela ORAS este radacina distribuirii regionale deoarece contine atributul “regiune”, astfel mapand catre cele 3 server-e locale.

ORAS(id_nume, nume_oras, regiune)

Pasul 1: Identificam predicatele simple folosite de aplicatie.

Analizăm cele mai folosite cereri (regula 80/20) pentru relația ORAS și, prin asta, am identificat următoarele predicate simple:

- p1: regiune = 'Transilvania'
- p2: regiune = 'Muntenia'
- p3: regiune = 'Moldova'

Primul set de predicate simple este: $Pr = \{ p1, p2, p3 \}$

Pasul 2: Aplicam algoritmul COMP_MIN

Algoritmul COMP_MIN găsește un subset complet și minimal $Pr' \subseteq Pr$.

- Inițializare: Luăm p1, care împarte tabela ORAS în două seturi distincte: orașe din Transilvania și orașe care nu sunt în Transilvania. Din moment ce cel puțin o aplicație accesează aceste seturi în mod diferit (de exemplu, o cerere listează sălile în funcție de regiune), p1 este relevant. Asta implică faptul că Pr' pentru moment este egal cu setul format din p1, iar noi obținem două fragmente: f1 și f1': $f1 = \sigma(\text{regiune}='Transilvania')(ORAS)$ și $f1' = \sigma(\text{regiune} \neq 'Transilvania')(ORAS)$.
- Iteratia 1: Luăm p2. Predicatul p2 va împărți fragmentul f1' (orașele care nu sunt în Transilvania) în orașe care sunt în Muntenia și orașe care sunt în Moldova. Din moment ce aplicațiile accesează aceste seturi diferit, p2 este relevant, iar acest lucru implică faptul că $Pr' = \{p1, p2\}$ și setul de fragmente va deveni $F = \{f1, f2, f3\}$, unde $f2 = \sigma(\text{regiune}='Muntenia')(ORAS)$ și $f3 = \sigma(\text{regiune}='Moldova')(ORAS)$.
- Iteratia 2: La următoarea iterație, vom lua p3, implicit reprezentat de $\neg p1 \wedge \neg p2$, deoarece domeniul pentru atributul regiune este Transilvania, Muntenia și Moldova. Dacă am adăuga p3 în setul de predicate, fragmentele nu s-ar mai diviza mai mult decât sunt în prezent. În acest caz, p3 nu este relevant și putem renunța la el.
- În concluzie, setul final de predicate simple este complet (fiecare fragment este accesat uniform de aplicație) și minimal (fiecare predicat este relevant) este reprezentat de $Pr' = \{p1, p2\}$.

Pasul 3: Aplicarea algoritmului PRIM_ORIZ.

Setul I de implicații este următorul:

- i1: $(\text{regiune} = 'Transilvania') \rightarrow \neg(\text{regiune} = 'Muntenia')$
- i2: $(\text{regiune} = 'Muntenia') \rightarrow \neg(\text{regiune} = 'Transilvania')$

Din $Pr' = \{p1, p2\}$ generam predicatele compuse M:

Predicat compus	Definitie	Fragment
m1	$regiune = 'Transilvania' \wedge regiune = 'Muntenia'$	Contradicție, il eliminăm
m2	$regiune = 'Transilvania' \wedge \neg(regiune = 'Muntenia')$	Echivalent cu $regiune = 'Transilvania'$
m3	$\neg(regiune = 'Transilvania') \wedge regiune = 'Muntenia'$	Echivalent cu $regiune = 'Muntenia'$
m4	$\neg(regiune = 'Transilvania') \wedge \neg(regiune = 'Muntenia')$	Echivalent cu $regiune = 'Moldova'$

Eliminam m1 deoarece este contradictor cu i1. Ramanem cu predicatele compuse m2,m3,m4.

Vom defini următorul semn de date ipotetice:

id_oras	nume_oras	regiune
1	Cluj-Napoca	Transilvania
2	Braşov	Transilvania
3	Bucureşti	Muntenia
4	Ploieşti	Muntenia
5	Iaşi	Moldova
6	Suceava	Moldova
7	Sibiu	Transilvania
8	Craiova	Muntenia

9	Bacău	Moldova
---	-------	---------

Obținem fragmentele:

ORAS_1 = $\sigma(\text{regiune} = \text{'Transilvania'})$ (ORAS); il salvam pe **Server S1**

id_oras	nume_oras	regiune
1	Cluj-Napoca	Transilvania
2	Braşov	Transilvania
7	Sibiu	Transilvania

ORAS_2 = $\sigma(\text{regiune} = \text{'Muntenia'})$ (ORAS); il salvam pe **Server S2**

id_oras	nume_oras	regiune
3	Bucureşti	Muntenia
4	Ploieşti	Muntenia
8	Craiova	Muntenia

ORAS_3 = $\sigma(\text{regiune} = \text{'Moldova'})$ (ORAS); il salvam pe **Server S3**

id_oras	nume_oras	regiune
5	Iaşi	Moldova
6	Suceava	Moldova
9	Bacău	Moldova

ii. Obținerea fragmentelor orizontale primare – obligatoriu

Relation	Fragment	Selection Formula	Server
ORAS	ORAS_1	$\sigma(\text{regiune} = \text{'Transilvania'})$ (ORAS)	S1
ORAS	ORAS_2	$\sigma(\text{regiune} = \text{'Muntenia'})$ (ORAS)	S2
ORAS	ORAS_3	$\sigma(\text{regiune} = \text{'Moldova'})$ (ORAS)	S3

b. (0,5p) Fragmentare orizontală derivată

Fragmentarea orizontală derivată este aplicată pe relația member a unei legături între două tabele, bazată pe fragmentarea relației owner. Având legătura L cu owner(L) = S și member(L) = R, fragmentele derivate sunt:

$$R_i = R \bowtie S_i, 1 \leq i \leq r$$

Propagam fragmentarea tabelului ORAS prin lantul de chei straine.

i. Obținerea fragmentelor orizontale derivate – obligatoriu

Fragmente derivate pentru tabela ADRESA:

Legatura L1: owner(L1) = ORAS, member(L1) = ADRESA (prin atributul id_oras)

- **ADRESA_1** = ADRESA $\times$ ORAS_1 - Toate adresele din orașele din Transilvania.
- **ADRESA_2** = ADRESA $\times$ ORAS_2 - Toate adresele din orașele din Muntenia.
- **ADRESA_3** = ADRESA $\times$ ORAS_3 - Toate adresele din orașele din Moldova.

Presupunem că avem următorul set de date ipotetic:

ADRESA (global):

id_adresa	id_oras	strada	numar
101	1	Str. Dorneasca	15
102	3	Bd. Unirii	25
103	5	Str. Vinului	7
104	2	Str. Raristea	15
105	4	Str. Democrației	3
106	6	Str. Albinelor	12

ADRESA_1 (Server S1): tupluri care au id_oras $\in \{1, 2, 7\} = \{101, 104\}$

ADRESA_2 (Server S2): tupluri care au id_oras $\in \{3, 4, 8\} = \{102, 105\}$

ADRESA_3 (Server S3): tupluri care au id_oras $\in \{5, 6, 9\} = \{103, 106\}$

Fragmente derivate pentru tabela SALA_DE_FITNESS

Legatura L2: owner(L2) = ADRESA, member(L2) = SALA_DE_FITNESS (prin atributul id_adresa)

- SALA_1 = SALA_DE_FITNESS $\times$ ADRESA_1 $\rightarrow$ Salile din Transilvania
- SALA_2 = SALA_DE_FITNESS $\times$ ADRESA_2 $\rightarrow$ Salile din Muntenia
- SALA_3 = SALA_DE_FITNESS $\times$ ADRESA_3 $\rightarrow$ Salile din Moldova

Presupunem că avem următorul set de date ipotetic:

id_sala	nume	id_adresa	email	telefon	denumire_plan
1	NoGym Cluj	101	nogym.cj@gym.ro	0264-111111	GOLD
2	SpartanGym București	102	spartangym.buc@gym.ro	021-222222	PLATINUM
3	MoldovaGym Iași	103	moldovagym.is@gym.ro	0232-333333	BRONZE
4	NoGym Brașov	104	nogym.bv@gym.ro	0268-444444	BRONZE
5	SpartanGym Ploiești	105	spartangym.pl@gym.ro	0244-555555	SILVER
6	MoldovaGym Suceava	106	moldovagym.sv@gym.ro	0230-666666	GOLD

SALA_1 (S1): {1, 4}; deoarece au id_adresa 101, 104

SALA_2 (S2): {2, 5}; deoarece au id_adresa 102, 105

SALA_3 (S3): {3, 6}; deoarece au id_adresa 103, 106

Fragmente derivate pentru tabela DEPARTAMENT

Legatur L3: owner(L3) = SALA_DE_FITNESS, member(L3) = DEPARTAMENT (prin atributul id_sala)

- DEPARTAMENT_1 = DEPARTAMENT $\times$ SALA_1
- DEPARTAMENT_2 = DEPARTAMENT $\times$ SALA_2
- DEPARTAMENT_3 = DEPARTAMENT $\times$ SALA_3

Pe serverul 1 vor sta departamentele care apartin salilor cu id 1 sau 4.

Pe serverul 2 vor sta departamentele care apartin salilor cu id 2 sau 5.

Pe serverul 3 vor sta departamentele care apartin salilor cu id 3 sau 6.

Fragmente derivate pentru tabela ANGAJAT

Legatura L4: owner(L4) = DEPARTAMENT, member(L4) = ANGAJAT (prin atributul id_departament)

- ANGAJAT_1 = ANGAJAT $\times$ DEPARTAMENT_1 $\rightarrow$ Angajatii din Transilvania
- ANGAJAT_2 = ANGAJAT $\times$ DEPARTAMENT_2 $\rightarrow$ Angajatii din Muntenia
- ANGAJAT_3 = ANGAJAT $\times$ DEPARTAMENT_3 $\rightarrow$ Angajatii din Moldova

Fragmente derivate pentru tabela APARAT_FITNESS

Legatura L5: owner(L5) = SALA_DE_FITNESS, member(L5) = APARAT_FITNESS (prin atributul id_sala)

- $APARAT_1 = APARAT_FITNESS \times SALA_1 \rightarrow$ aparatele salilor de fitness din Transilvania
- $APARAT_2 = APARAT_FITNESS \times SALA_2 \rightarrow$ aparatele salilor de fitness din Muntenia
- $APARAT_3 = APARAT_FITNESS \times SALA_3 \rightarrow$ aparatele salilor de fitness din Moldova

Fragmente derivate pentru tabela CLASE_DE_FITNESS

Legatura L6: owner(L6) = SALA_DE_FITNESS, member(L6) = CLASE_DE_FITNESS (prin atributul id_sala)

- $CLASE_1 = CLASE_DE_FITNESS \times SALA_1 \rightarrow$ clasele de fitness pentru salile din Transilvania
- $CLASE_2 = CLASE_DE_FITNESS \times SALA_2 \rightarrow$ clasele de fitness pentru salile din Muntenia
- $CLASE_3 = CLASE_DE_FITNESS \times SALA_3 \rightarrow$ clasele de fitness pentru salile din Moldova

Fragmente derivate pentru tabela EVENIMENT

Legatura L7: owner(L7) = SALA_DE_FITNESS, member(L7) = EVENIMENT (prin atributul id_sala)

- $EVENIMENT_1 = EVENIMENT \times SALA_1 \rightarrow$ evenimentele pentru salile din Transilvania
- $EVENIMENT_2 = EVENIMENT \times SALA_2 \rightarrow$ evenimentele pentru salile din Muntenia
- $EVENIMENT_3 = EVENIMENT \times SALA_3 \rightarrow$ evenimentele pentru salile din Moldova

Fragmente derivate pentru tabela BILET

Legatura L8: owner(L8) = EVENIMENT, member(L8) = BILET (prin atributul id_eveniment)

- BILET_1 = BILET $\times$ EVENIMENT_1 $\rightarrow$ biletele pentru evenimentele din salile din Transilvania
- BILET_2 = BILET $\times$ EVENIMENT_2 $\rightarrow$ biletele pentru evenimentele din salile din Muntenia
- BILET_3 = BILET $\times$ EVENIMENT_3 $\rightarrow$ biletele pentru evenimentele din salile din Moldova

Fragmente derivate pentru tabela PRODUS_COMERCIAL

Legatura L9: owner(L9) = SALA_DE_FITNESS, member(L9) = PRODUS_COMERCIAL (prin atributul id_sala)

- PRODUS_1 = PRODUS_COMERCIAL $\times$ SALA_1 $\rightarrow$ produsele pentru salile din Transilvania
- PRODUS_2 = PRODUS_COMERCIAL $\times$ SALA_2 $\rightarrow$ produsele pentru salile din Muntenia
- PRODUS_3 = PRODUS_COMERCIAL $\times$ SALA_3 $\rightarrow$ produsele pentru salile din Moldova

Fragmente derivate pentru tabela VINDE

Legatura L10: owner(L10) = SALA_DE_FITNESS, member(L10) = VINDE (prin atributul id_sala)

- VINDE_1 = VINDE $\times$ SALA_1 $\rightarrow$ intrarile din tabela VINDE pentru salile din Transilvania
- VINDE_2 = VINDE $\times$ SALA_2 $\rightarrow$ intrarile din tabela VINDE pentru salile din Muntenia
- VINDE_3 = VINDE $\times$ SALA_3 $\rightarrow$ intrarile din tabela VINDE pentru salile din Moldova

Fragmente derivate pentru tabela ABONAMENT

Am decis să nu fragmentăm pe orizontală tabela ABONAMENT, deoarece, fiind condiționată de tabela PLAN, subscripțiile către sălile noastre oferă clienților acces în

diferite sali, în funcție de abonamentul și planul lor. Un client poate avea un plan care să îi ofere posibilitatea de a merge la săli din diferite regiuni.

Astfel, dacă am continua cu fragmentarea pe cele trei servere locale ale noastre cu tabela ABONAMENT, există posibilitatea ca abonamentele cu planuri premium să fie duplicate pe toate cele trei servere locale ale noastre sau pe câte două dintre ele.

Fragmente derivate pentru tabela CLIENT

Dat fiind faptul că un client poate să dețină un abonament, care îi oferă acces la mai multe regiuni, am decis că tabela CLIENT să nu fie fragmentată deoarece există posibilitatea ca clienții care dețin abonamente cu planuri premium să fie duplicați pe toate cele trei servere locale ale noastre sau pe câte două dintre ele.

Fragmente derivate pentru tabela PLAN

În cazul tabelii PLAN, aceasta va avea mereu 4 tupluri, pentru orice regiune. Astfel, dacă am încerca să fragmentăm, v-a trebui să punem toate aceste 4 tupluri pe fiecare server, astfel creându-se duplicate și contrazicându-se cu scopul fragmentării. Din această cauză, am decis să nu fragmentăm tabela PLAN.

c. (1p) Fragmentare verticală

Fragmentarea verticală împarte o tabelă în mai multe coloane, fiecare fragmentare conținând cheia primară a tabelului.

Am aplicat fragmentarea verticală pe tabela ANGAJAT, pe fiecare fragment obținut prin fragmentare derivată orizontală, și vom obține astfel două fragmentări verticale pe fiecare fragmentare orizontală derivată.

ANGAJAT_i(id_angajat, nume, prenume, email, telefon, salariu, id_departament, functie) - reprezintă indecele fiecărui fragment obținut prin fragmentare orizontală derivată.

Cheia primară a tabelului este id_angajat. Aceasta va fi replicată în fiecare fragment vertical. Atributele care nu sunt cheie primară sunt următoarele:

- A1 = nume
- A2 = prenume

- A3 = email
- A4 = telefon
- A5 = salariu
- A6 = id_departament
- A7 = functie

i. Aplicarea algoritmului de fragmentare verticală (exemplificarea pașilor algoritmului pe baza unor date ipotetice)

Pasul 1: Identificam 4 aplicatii reprezentative (query-uri) care opereaza pe tabela ANGAJAT si definim matricea VA:

q1: datelele necesare departamentului de HR

“SELECT nume, prenume, email, telefon FROM ANGAJAT WHERE ...”

q2: Procesarea platilor salariilor

“SELECT salariu, id_departament, functie FROM ANGAJAT WHERE ...”

q3: Informatii de contact pentru angajati

“SELECT email, telefon FROM ANGAJAT WHERE ...”

q4: Rapoarte legate de detaliile angajatiilor dintr-un departament

“SELECT nume, prenume, id_departament, functie FROM ANGAJAT WHERE ...”

Valorile folosirii atributelor, matricea VA(unde $VA(i,j) = use(q_i, A_j) \rightarrow$ functia_identitate(atributulu A_J este folosit in aplicatia q_i)

A1(nume)	A2(prenume)	A3(email)	A4(telefon)	A5(salariu)	A6(id_departament)	A7(functie)	
q1	1	1	1	1	0	0	0

q2	0	0	0	0	1	1	1
q3	0	0	1	1	0	0	0
q4	1	1	0	0	0	1	1

Pasul 2: Calcularea Attribute Affinity Matrix (AA)

Reteaua are 3 statii si $ref_l(q_k) = 1$ for all k, l .

Presupunem că frecvențele de acces ale aplicațiilor sunt următoarele:

Station 1	Station 2	Station 3	Total	
q1	15	20	10	45
q2	25	15	20	60
q3	10	5	5	20
q4	5	10	15	30

Afinitatea dintre atributul A_i si atributul A_j este:

$$\text{aff}(A_i, A_j) = \sum_{k \in K} \sum_l \text{acc}_l(q_k)$$

$$\text{Unde } K = \{k \mid \text{use}(q_k, A_i) = 1 \text{ AND } \text{use}(q_k, A_j) = 1\}$$

In urmatoarele randuri vom calcula valorile matricei de afinitate:

1. $\text{aff}(A_1, A_1): K=\{q_1, q_4\} \rightarrow 45+30 = 75$
2. $\text{aff}(A_1, A_2): K=\{q_1, q_4\} \rightarrow 45+30 = 75$
3. $\text{aff}(A_1, A_3): K=\{q_1\} \rightarrow 45$
4. $\text{aff}(A_1, A_4): K=\{q_1\} \rightarrow 45$
5. $\text{aff}(A_1, A_5): K=\{\} \rightarrow 0$
6. $\text{aff}(A_1, A_6): K=\{q_4\} \rightarrow 30$
7. $\text{aff}(A_1, A_7): K=\{q_4\} \rightarrow 30$
8. $\text{aff}(A_2, A_2): K=\{q_1, q_4\} \rightarrow 45+30 = 75$
9. $\text{aff}(A_2, A_3): K=\{q_1\} \rightarrow 45$
10. $\text{aff}(A_2, A_4): K=\{q_1\} \rightarrow 45$
11. $\text{aff}(A_2, A_5): K=\{\} \rightarrow 0$
12. $\text{aff}(A_2, A_6): K=\{q_4\} \rightarrow 30$
13. $\text{aff}(A_2, A_7): K=\{q_4\} \rightarrow 30$
14. $\text{aff}(A_3, A_3): K=\{q_1, q_3\} \rightarrow 45+20 = 65$
15. $\text{aff}(A_3, A_4): K=\{q_1, q_3\} \rightarrow 45+20 = 65$
16. $\text{aff}(A_3, A_5): K=\{\} \rightarrow 0$
17. $\text{aff}(A_3, A_6): K=\{\} \rightarrow 0$
18. $\text{aff}(A_3, A_7): K=\{\} \rightarrow 0$
19. $\text{aff}(A_4, A_4): K=\{q_1, q_3\} \rightarrow 45+20 = 65$
20. $\text{aff}(A_4, A_5): K=\{\} \rightarrow 0$
21. $\text{aff}(A_4, A_6): K=\{\} \rightarrow 0$
22. $\text{aff}(A_4, A_7): K=\{\} \rightarrow 0$
23. $\text{aff}(A_5, A_5): K=\{q_2\} \rightarrow 60$
24. $\text{aff}(A_5, A_6): K=\{q_2\} \rightarrow 60$
25. $\text{aff}(A_5, A_7): K=\{q_2\} \rightarrow 60$
26. $\text{aff}(A_6, A_6): K=\{q_2, q_4\} \rightarrow 60+30 = 90$
27. $\text{aff}(A_6, A_7): K=\{q_2, q_4\} \rightarrow 60+30 = 90$
28. $\text{aff}(A_7, A_7): K=\{q_2, q_4\} \rightarrow 60+30 = 90$

Attribute Affinity Matrix AA

	A1	A2	A3	A4	A5	A6	A7
A1	75	75	45	45	0	30	30
A2	75	75	45	45	0	30	30
A3	45	45	65	65	0	0	0
A4	45	45	65	65	0	0	0
A5	0	0	0	0	60	60	60
A6	30	30	0	0	60	90	90
A7	30	30	0	0	60	90	90

Pas 3: Aplicam algoritmul LEG_AFIN

Algoritmul permuta randurile si coloanele a AA pentru a grupa atributele cu afinitate mare. Astfel se produce matrice Clustered Affinity CA.

Initializare: Punem coloanaele A1 si A2 in CA, la random

Iteratie: punem A3

Avem $\text{cont}(A_i, A_3, A_j) = 2 \cdot \text{bond}(A_i, A_3) + 2 \cdot \text{bond}(A_3, A_j) - 2 \cdot \text{bond}(A_i, A_j)$ pentru toate pozitiile posibile

Prima data calculam valorile de legatura, $\text{bond}(A_x, A_y) = \sum_i \text{aff}(A_i, A_x) \cdot \text{aff}(A_i, A_y)$:

$$\text{bond}(A_1, A_2) = 75 \cdot 75 + 75 \cdot 75 + 45 \cdot 45 + 45 \cdot 45 + 0 \cdot 0 + 30 \cdot 30 + 30 \cdot 30 = 5625 + 5625 + 2025 + 2025 + 0 + 900 + 900 = 17100$$

$$\text{bond}(A_1, A_3) = 75 \cdot 45 + 75 \cdot 45 + 45 \cdot 65 + 45 \cdot 65 + 0 \cdot 0 + 30 \cdot 0 + 30 \cdot 0 = 3375 + 3375 + 2925 + 2925 + 0 + 0 + 0 = 12600$$

$$\text{bond}(A_2, A_3) = 75 \cdot 45 + 75 \cdot 45 + 45 \cdot 65 + 45 \cdot 65 + 0 \cdot 0 + 30 \cdot 0 + 30 \cdot 0 = 12600$$

$$\text{bond}(A_3, A_3) = 45^2 + 45^2 + 65^2 + 65^2 + 0 + 0 + 0 = 2025 + 2025 + 4225 + 4225 = 12500$$

Mai departe calculam plasarea coloanei A3 cu respect la coloanele A1 si A2.

$$\text{Inainte } A_1: \text{cont}(A_0, A_3, A_1) = 2 \cdot \text{bond}(A_0, A_3) + 2 \cdot \text{bond}(A_3, A_1) - 2 \cdot \text{bond}(A_0, A_1) = 0 + 2 \cdot 12600 - 0 = 25200$$

$$\text{Intre } A_1 \text{ si } A_2: \text{cont}(A_1, A_3, A_2) = 2 \cdot \text{bond}(A_1, A_3) + 2 \cdot \text{bond}(A_3, A_2) - 2 \cdot \text{bond}(A_1, A_2) = 2 \cdot 12600 + 2 \cdot 12600 - 2 \cdot 17100 = 50400 - 34200 = 16200$$

$$\text{Dupa } A_2: \text{cont}(A_2, A_3, A_0) = 2 \cdot \text{bond}(A_2, A_3) + 0 - 0 = 25200$$

Cel mai mult obtinem inainte de A1 si dupa A2 --> decidem sa o punem dupa A2

Iteratie: Punem A4

$$\text{bond}(A_3, A_4) = 45 \cdot 45 + 45 \cdot 45 + 65 \cdot 65 + 65 \cdot 65 + 0 \cdot 0 + 0 \cdot 0 + 0 \cdot 0 = 2025 + 2025 + 4225 + 4225 = 12500$$

$$\text{bond}(A_1, A_4) = 75 \cdot 45 + 75 \cdot 45 + 45 \cdot 65 + 45 \cdot 65 + 0 \cdot 0 + 30 \cdot 0 + 30 \cdot 0 = 12600$$

$$\text{bond}(A_2, A_4) = 12600$$

Mai departe calculam cel mai bun amplasament pentru A4:

$$\text{Inainte } A_1: \text{cont}(A_0, A_4, A_1) = 0 + 2 \cdot 12600 - 0 = 25200$$

$$\text{Intre } A_1 \text{ si } A_2: \text{cont}(A_1, A_4, A_2) = 2 \cdot 12600 + 2 \cdot 12600 - 2 \cdot 17100 = 16200$$

Intre A2 si A3: $\text{cont}(A2, A4, A3) = 2 \cdot 12600 + 2 \cdot 12500 - 2 \cdot 12600 = 25000$

Intre A3: $\text{cont}(A3, A4, A0) = 2 \cdot 12500 + 0 - 0 = 25000$

Cel mai mult obtinem daca o punem inainte de A1, 25200. Asadar punem A4 inainte de A1

$\Rightarrow A4, A1, A2, A3$

Iteratie: Punem A5

$\text{bond}(A4, A5) = 45 \cdot 0 + 45 \cdot 0 + 65 \cdot 0 + 65 \cdot 0 + 0 \cdot 60 + 0 \cdot 60 + 0 \cdot 60 = 0$

$\text{bond}(A1, A5) = 75 \cdot 0 + 75 \cdot 0 + 45 \cdot 0 + 45 \cdot 0 + 0 \cdot 60 + 30 \cdot 60 + 30 \cdot 60 = 3600$

$\text{bond}(A2, A5) = 3600$

$\text{bond}(A3, A5) = 0$

Mai departe calculam cel mai bun amplasament pentru A4:

Inainte A4: $\text{cont}(A0, A5, A4) = 0 + 2 \cdot 0 - 0 = 0$

Intre A4 si A1: $\text{cont}(A4, A5, A1) = 2 \cdot 0 + 2 \cdot 3600 - 2 \cdot 12600 = -18000$

Intre A1 si A2: $\text{cont}(A1, A5, A2) = 2 \cdot 3600 + 2 \cdot 3600 - 2 \cdot 17100 = -19800$

Intre A2 si A3: $\text{cont}(A2, A5, A3) = 2 \cdot 3600 + 2 \cdot 0 - 2 \cdot 12600 = -18000$

Dupa A3: $\text{cont}(A3, A5, A0) = 2 \cdot 0 + 0 - 0 = 0$

Cel mai mult obtinem daca o punem dupa A3, 0.

$\Rightarrow A4, A1, A2, A3, A5$

Iteratie: Punem A6

$\text{bond}(A5, A6) = 0 \cdot 30 + 0 \cdot 30 + 0 \cdot 0 + 0 \cdot 0 + 60 \cdot 60 + 60 \cdot 90 + 60 \cdot 90 = 3600 + 5400 + 5400 = 14400$

$$\text{bond}(A3, A6) = 45 \cdot 30 + 45 \cdot 30 + 65 \cdot 0 + 65 \cdot 0 + 0 \cdot 60 + 0 \cdot 90 + 0 \cdot 90 = 1350 + 1350 = 2700$$

$$\text{bond}(A4, A6) = 45 \cdot 30 + 45 \cdot 30 + 65 \cdot 0 + 65 \cdot 0 + 0 \cdot 60 + 0 \cdot 90 + 0 \cdot 90 = 2700$$

$$\begin{aligned} \text{bond}(A1, A6) &= 75 \cdot 30 + 75 \cdot 30 + 45 \cdot 0 + 45 \cdot 0 + 0 \cdot 60 + 30 \cdot 90 + 30 \cdot 90 = \\ &2250 + 2250 + 2700 + 2700 = 9900 \end{aligned}$$

$$\text{bond}(A2, A6) = 9900$$

$$\text{Inainte A4: cont}(A0, A6, A4) = 0 + 2 \cdot 2700 - 0 = 5400$$

$$\text{Intre A4 si A1: cont}(A4, A6, A1) = 2 \cdot 2700 + 2 \cdot 9900 - 2 \cdot 12600 = 0$$

$$\text{Intre A1 si A2: cont}(A1, A6, A2) = 2 \cdot 9900 + 2 \cdot 9900 - 2 \cdot 17100 = 5400$$

$$\text{Intre A2 si A3: cont}(A2, A6, A3) = 2 \cdot 9900 + 2 \cdot 2700 - 2 \cdot 12600 = 0$$

$$\text{Intre A3 si A5: cont}(A3, A6, A5) = 2 \cdot 2700 + 2 \cdot 14400 - 0 = 34200$$

$$\text{Dupa A5: cont}(A5, A6, A0) = 2 \cdot 14400 + 2 \cdot 0 - 2 \cdot 0 = 28,800$$

Cel mai bine (34200) o plasam intre A3 si A5

== > A4, A1, A2, A3, A6, A5

Iteratie: Plasam A7

$$\begin{aligned} \text{bond}(A6, A7) &= 30 \cdot 30 + 30 \cdot 30 + 0 \cdot 0 + 0 \cdot 0 + 60 \cdot 60 + 90 \cdot 90 + 90 \cdot 90 = \\ &900 + 900 + 3600 + 8100 + 8100 = 21600 \end{aligned}$$

$$\text{bond}(A5, A7) = 0 \cdot 30 + 0 \cdot 30 + 0 \cdot 0 + 0 \cdot 0 + 60 \cdot 60 + 60 \cdot 90 + 60 \cdot 90 = 3600 + 5400 + 5400 = 14400$$

$$\text{Bond}(A4, A7) = 45 \cdot 30 + 45 \cdot 30 = 2,700$$

$$\text{Bond}(A3, A7) = 45 \cdot 30 + 45 \cdot 30 = 2,700$$

$$\text{Bond}(A2, A7) = 75 \cdot 30 + 75 \cdot 30 + 30 \cdot 90 + 30 \cdot 90 = 9900$$

$$\text{Bond}(A1, A7) = 75 \cdot 30 + 75 \cdot 30 + 30 \cdot 90 + 30 \cdot 90 = 9900$$

$$\text{Inainte A4: cont}(A0, A7, A4) = 0 + 2 \cdot 2700 - 0 = 5400$$

Intre A4 si A1: $\text{cont}(A4, A7, A1) = 2 \cdot 2700 + 2 \cdot 9900 - 2 \cdot 12600 = 0$

Intre A1 si A2: $\text{cont}(A1, A7, A2) = 2 \cdot 9900 + 2 \cdot 9900 - 2 \cdot 17100 = 5400$

Intre A2 si A3: $\text{cont}(A2, A7, A3) = 2 \cdot 9900 + 2 \cdot 2700 - 2 \cdot 12600 = 0$

Intre A3 si A6: $\text{cont}(A3, A7, A6) = 2 \cdot 2700 + 2 \cdot 14400 - 2 \cdot 2700 = 28,800$

Intre A6 si A5: $\text{cont}(A6, A7, A5) = 2 \cdot 21600 + 2 \cdot 14400 - 2 \cdot 14400 = 43200$

Dupa A5: $\text{cont}(A5, A7, A0) = 2 \cdot 14400$

Cel mai bine intre A6 si A5.

$\Rightarrow A4, A1, A2, A3, A6, A7, A5$

Obtinem Clustered Affinity Matrix CA:

	A4	A1	A2	A3	A6	A7	A5
A4	65	45	45	65	0	0	0
A1	45	75	75	45	30	30	0
A2	45	75	75	45	30	30	0
A3	65	45	45	65	0	0	0
A6	0	30	30	0	90	90	60
A7	0	30	30	0	90	90	60
A5	0	0	0	0	60	60	60

Aplicam algoritmul PART

Vom calcula configuratia ce maximizeaza $z = CTQ \cdot CBQ - COQ^2$

Matricea CA a noastra are urmatoarele coloane in ordine {A4, A1, A2, A3, A6, A7, A5}

Vom pune punctul i pe diagonala principala si obtinem:

TA={A4, A1, A2, A3, A6, A7}

BA={A5}

Q1 foloseste {A1,A2,A3, A4} apartin TA

Q2 foloseste {A5, A6,A7} apartin TA si BA

Q3 foloseste {A3,A4} apartin TA

Q4 foloseste {A1,A2,A6,A7} apartin TA

=> TQ={Q1, Q3, Q4}

BQ= NIMIC

OQ= {Q2}

CTQ= 45+20+30=95

CBQ= 0

COQ=60

Z=-60*60= -3600

Max_z = -3600

Urmatoare iteratie:

TA={A4, A1, A2, A3, A6}

BA={A7, A5}

$TQ=\{Q1, Q3\}$

$BQ= \text{NIMIC}$

$OQ=\{Q2, Q4\}$

$Z= (45+20)*0 - (60+30)**2= -8100$

Max_z ramane la fel

Urmatoare iteratie:

$TA=\{A4, A1, A2, A3\}$

$BA=\{A6, A7, A5\}$

$TQ=\{Q1, Q3\}$

$BQ=\{Q2\}$

$OQ=\{Q4\}$

$Z=65*60-30**2=3000$

MAX_Z devine 3000 cu config asta

Urmatoare iteratie:

$TA=\{A4, A1, A2\}$

$BA=\{A3, A6, A7, A5\}$

$TQ=\{\}$

$BQ=\{Q2\}$

$OQ=\{Q1, Q3, Q4\}$

$Z=-95**2$

Max_z ramane la fel

Urmatoare iteratie:

$TA=\{A4, A1\}$

$BA=\{A2, A3, A6, A7, A5\}$

$TQ=\{\}$

$BQ=\{Q2\}$

$OQ=\{Q1, Q3, Q4\}$

Z e negativ, la fel

Urmatoare iteratie:

$TA=\{A4\}$

$BA=\{A1, A2, A3, A6, A7, A5\}$

$TQ=\{\}$

$BQ=\{Q2, Q4\}$

$OQ=\{Q1, Q3\}$

Z e negativ

Dupa aceste iteratii c. m. buna configuratie (c.m. mare z) obtine din

$TA=\{A4, A1, A2, A3\}$

$BA=\{A6, A7, A5\}$

Cu $Z=3000$

Aplicam SCHIMB:

Obtinem A1, A2, A3, A6, A7, A5, A4,

Prima iteratie:

$TA=\{A1, A2, A3, A6, A7, A5\}$

$BA=\{A4\}$

$TQ=\{Q2, Q4\}$

$BQ=\{\}$

$OQ=\{Q1, Q3\}$

Obtinem z negativ

Urmatoare Iteratie:

$TA=\{A1, A2, A3, A6, A7\}$

$BA=\{A5, A4\}$

$TQ=\{ Q4\}$

$BQ=\{\}$

$OQ=\{Q1, Q3, Q2\}$

Z negativ

Urmatoare Iteratie:

$TA=\{A1, A2, A3, A6\}$

$BA=\{A7, A5, A4\}$

$TQ=\{\}$

$BQ=\{\}$

$OQ=\{Q1, Q3, Q2, Q4\}$

Z negativ

Urmatoare Iteratie:

$TA=\{A1, A2, A3\}$

$BA=\{A6, A7, A5, A4\}$

$TQ=\{\}$

$BQ=\{Q2\}$

$OQ=\{Q1, Q3, Q4\}$

Z negativ

Urmatoare Iteratie:

TA={A1, A2}

BA={A3, A6, A7, A5, A4}

TQ={}

BQ={Q2, Q3}

OQ={Q1, Q4}

Z negativ

Urmatoare Iteratie:

TA={A1}

BA={A2, A3, A6, A7, A5, A4}

TQ={}

BQ={Q2, Q3}

OQ={Q1, Q4}

Z negativ

Dupa aceste iteratii c. m. buna configuratie (c.m. mare z) ramane

TA={A4, A1, A2, A3}

BA={A6 , A7, A5}

Cu Z=3000

Obtinem fragmentare verticala urmatoare:

ANGAJAT_V1(id_angajat, telefon, nume, prenume, email)

Si

ANGAJAT_V2(id_angajat, id_departament, functie, salariu)

5. (1p) Verificarea corectitudinii fragmentărilor realizate

Verificarea fragmentării primare orizontale a tabeli ORAS

Completitudine:

Având în vedere că orice tuplu din tabela ORAS are exact o valoare regiune $\in \{\text{Transilvania, Muntenia, Moldova}\}$, cele trei fragmente pe care le-am definit mai sus includ cele trei valori. Fiecare tuplu din tabela ORAS este prezent într-un fragment.

Din moment ce algoritmul aplicat pentru derivarea orizontală garantează că obținem un set de predicate complet și minimal, tragem concluzia că această fragmentare orizontală primară satisface cerința de completitudine.

Reconstrucția:

Tabela original poate fi obținută din reuniunea celor 3 fragmente (date de cele 3 regiuni)

$ORAS = ORAS_1 \cup ORAS_2 \cup ORAS_3$

Din moment ce toate cele trei fragmente sunt definite de o selecție din tabela originală, fiecare subset fiind distinct, uniunea acestor fragmente reproduce întocmai tabela originală.

Disjuncție:

Selecția de predicate este mutual exclusivă, deoarece un oraș poate aparține unei singure regiuni. Prin urmare, un tuplu apare în exact unul dintre cele trei fragmente.

$ORAS_i \cap ORAS_j = \emptyset, \text{ for all } i \neq j$

Verificarea fragmentării derivate orizontale

Completitudine:

Completitudinea fragmentării orizontale derivate stă la baza integrității referențiale. Adică, pentru orice relație R member, având o cheie străină care referențiază tabela owner S:

Orice tuplu din R are un tuplu corespondent în tabela S, datorită constrângerii de identitate garantate de cheia străină.

Din moment ce relația este complet fragmentată pe orizontală (dată de completitudinea tabelii Owner), fiecare tuplu din R este referențiat printr-un join la tabela Member S.

Luăm ca exemplu tabela ADRESA ce are o cheie străină **id_oras** care referențiază tabela ORAS.

Fiecare tuplu din tabela **ORAS** are o cheie primară **id_oras**. Iar aceste tupluri sunt împartite pe cele 3 fragmente.

Prin urmare, fiecare tuplu din tabela ADRESA apare la rândul ei într-unul dintre cele trei fragmente ale sale.

Prin aceeași logică de referințiere pe baza cheii străine, deducem că toate fragmentele derivate satisfac cerința de completitudine.

Reconstructia:

Tabela original poate fi obținută din reuniunea celor 3 fragmente (date de cele 3 regiuni). Această afirmație este valabilă pentru tabela Owner, dar, dat fiind faptul constrângerii cheii străine, este valabilă și pentru tabelele Member.

$R = R_1 \cup R_2 \cup R_3$, pentru toate R – relațiile derivate fragmentate pe orizontală.

Din moment ce toate cele trei fragmente sunt definite de o selecție din tabela originală, fiecare subset fiind distinct, uniunea acestor fragmente reproduce întocmai tabela originală.

Disjunctia:

Constrângerea de disjuncție, este satisfăcută având în vedere disjuncția tabelii Member ORAS și constrângerea de integritate referențială dată de cheia străină, ce leagă tabela Oraș de celelalte tabele derivate fragmentate.

Selecția de predicate este mutual exclusivă, deoarece un oraș poate aparține unei singure regiuni. Prin urmare, un tuplu apare în exact unul dintre cele trei fragmente.

$R_i \cap R_j = \emptyset$, for all $i \neq j$, pentru toate relațiile R fragmentate derivate pe orizontală

Verificarea fragmentării verticale

Completitudinea:

Mulțimea atributelor din ANGAJAT_V1 și ANGAJAT_V2 împreună acoperă toate attributele ANGAJAT: $\{id_angajat, nume, prenume, email, telefon\} \cup \{id_angajat, salariu, id_departament, functie\} = \{id_angajat, nume, prenume, email, telefon, salariu, id_departament, functie\}$. Aceasta este egală cu mulțimea completă de attribute a ANGAJAT.

Reconstrucția:

Reconstrucția relației originale se poate obține prin compunerea celor două fragmente obținute prin derivare pe verticală. Compunerea se face prin cheia primară $id_angajat$.

$ANGAJAT = ANGAJAT_V1 \bowtie (id_angajat) ANGAJAT_V2$

$id_angajat$ este cheia primară și este regăsită în ambele fragmente, deci compunerea va obține tabela originală.

Disjunctia:

Pentru fragmentarea verticală, disjuncția este definită doar pentru attributele non-cheie primare. Cheia primară $id_angajat$ este intenționat replicată în ambele fragmente. Attributele rămase sunt:

- ANGAJAT_V1 non-cheie primară: $\{nume, prenume, email, telefon\}$
- ANGAJAT_V2 non-cheie primară: $\{salariu, id_departament, functie\}$

Aceste mulțimi sunt disjuncte: $\{nume, prenume, email, telefon\} \cap \{salariu, id_departament, functie\} = \emptyset$

6. (0,5p) Argumentarea deciziei de replicare a anumitor relații sau/și de stocare a unei relații pe o singură stație

Relatii replicate:

1. Plan

Tabela este de dimensiuni mici si va contine mereu doar patru tupluri. Aceasta este referențiată de tabela ABONAMENT prin cheia străină denumire_plan. Avand in vedere ca aceasta table este read-only, nicio schimbare nu se va mai face pe acest tabel, si este un tabel de dimensiuni mici, va fi replicat pe toate cele trei servere.

2. Client

Un client nu este legat de o singură regiune. Clienții pot cumpăra abonamente care vor oferi acces la săli din regiuni diferite. Pe aceasta relatie, query-urile frecvente vor fi cele de adaugare a unui nou client. Nu vor fi schimbari recurente pe acest tabel. Asadar, tabela CLIENT va fi replicata pe toate serverele locale, fiind o tabela de dimensiuni moderate, pe care nu se aplica schimbari recurente. Singurele verificari recurente vor fi daca clientul exista, deci query-uri read-only.

Relatie stocata pe serverul global:

1. Abonament

Un abonament ofera intrari la diferite sali din diferite regiuni. Un client isi va cumpara un abonament nou in fiecare luna. Dat fiind ca toti clientii vor realiza aceasta operatie recurenta, luna de luna, am decis ca tabela ABONAMENT sa fie stocata doar pe serverul global. Astfel, evitam multiple query-uri de INSERT facute recurent pe toate severele locale.

7. (0,75p) Crearea schemelor conceptuale locale (corespunzătoare bazelor de date locale) - obligatoriu

7.1. Schema conceptuala locala — Server S1 (Transilvania)

Tabel	Tip	Coloane
ORAS_1	Fragment (FOP)	id_oras, nume_oras, regiune

ADRESA_1	Fragment (FOD)	id_adresa, id_oras, strada, numar
SALA_1	Fragment (FOD)	id_sala, nume, id_adresa, email, telefon, denumire_plan
DEPARTAMENT_1	Fragment (FOD)	id_departament, nume, salariu_minim, salariu_maxim, id_sala
ANGAJAT_V1_1	Fragment (FOD+FV)	id_angajat, telefon, nume, prenume, email
ANGAJAT_V2_1	Fragment (FOD+FV)	id_angajat, id_departament, functie, salariu
APARAT_1	Fragment (FOD)	id_aparat, marca, denumire, data_productie, grupa_musculara, id_sala
CLASE_1	Fragment (FOD)	id_clasa, denumire, categorie, id_sala
EVENIMENT_1	Fragment (FOD)	id_eveniment, id_sala, nume, data, numar_bilete
BILET_1	Fragment (FOD)	id_bilet, id_eveniment, pret, id_client
PRODUS_1	Fragment (FOD)	id_produs, denumire, categorie, pret, data_expirare, id_sala, id_client
VINDE_1	Fragment (FOD)	id_sala, id_abonament
CLIENT	Replicat	id_client, nume, prenume, email, telefon
PLAN	Replicat	denumire_plan, pret

7.1. Schema conceptuala locala — Server S2 (Muntenia)

Tabel	Tip	Coloane
ORAS_2	Fragment (FOP)	id_oras, nume_oras, regiune
ADRESA_2	Fragment (FOD)	id_adresa, id_oras, strada, numar

SALA_2	Fragment (FOD)	id_sala, nume, id_adresa, email, telefon, denumire_plan
DEPARTAMENT_2	Fragment (FOD)	id_departament, nume, salariu_minim, salariu_maxim, id_sala
ANGAJAT_V1_2	Fragment (FOD+FV)	id_angajat, telefon, nume, prenume, email
ANGAJAT_V2_2	Fragment (FOD+FV)	id_angajat, id_departament, functie, salariu
APARAT_2	Fragment (FOD)	id_aparat, marca, denumire, data_productie, grupa_musculara, id_sala
CLASE_2	Fragment (FOD)	id_clasa, denumire, categorie, id_sala
EVENIMENT_2	Fragment (FOD)	id_eveniment, id_sala, nume, data, numar_bilete
BILET_2	Fragment (FOD)	id_bilet, id_eveniment, pret, id_client
PRODUS_2	Fragment (FOD)	id_produs, denumire, categorie, pret, data_expirare, id_sala, id_client
VINDE_2	Fragment (FOD)	id_sala, id_abonament
CLIENT	Replicat	id_client, nume, prenume, email, telefon
PLAN	Replicat	denumire_plan, pret

7.1. Schema conceptuala locala — Server S3 (Moldova)

Tabel	Tip	Coloane
ORAS_3	Fragment (FOP)	id_oras, nume_oras, regiune
ADRESA_3	Fragment (FOD)	id_adresa, id_oras, strada, numar

SALA_3	Fragment (FOD)	id_sala, nume, id_adresa, email, telefon, denumire_plan
DEPARTAMENT_3	Fragment (FOD)	id_departament, nume, salariu_minim, salariu_maxim, id_sala
ANGAJAT_V1_3	Fragment (FOD+FV)	id_angajat, telefon, nume, prenume, email
ANGAJAT_V2_3	Fragment (FOD+FV)	id_angajat, id_departament, functie, salariu
APARAT_3	Fragment (FOD)	id_aparat, marca, denumire, data_productie, grupa_musculara, id_sala
CLASE_3	Fragment (FOD)	id_clasa, denumire, categorie, id_sala
EVENIMENT_3	Fragment (FOD)	id_eveniment, id_sala, nume, data, numar_bilete
BILET_3	Fragment (FOD)	id_bilet, id_eveniment, pret, id_client
PRODUS_3	Fragment (FOD)	id_produs, denumire, categorie, pret, data_expirare, id_sala, id_client
VINDE_3	Fragment (FOD)	id_sala, id_abonament
CLIENT	Replicat	id_client, nume, prenume, email, telefon
PLAN	Replicat	denumire_plan, pret

8. (2p) Lista tuturor constrângerilor ce trebuie îndeplinite mde model - obligatoriu

a. (0,5p) Unicitate

i. unicitate locală

Fiecare fragment local trebuie sa aibe unicitate pe propriile date:

Tabela	Constragere de unicitate	Tip
ORAS_i	id_oras este unic	Local per fragment
ADRESA_i	id_adresa este unic	Local per fragment
SALA_i	id_sala este unic	Local per fragment
DEPARTAMENT_i	id_departament este unic	Local per fragment
ANGAJAT_V1_i	id_angajat este unic	Local per fragment
ANGAJAT_V2_i	id_angajat este unic	Local per fragment
APARAT_i	id_aparat este unic	Local per fragment
CLASE_i	id_clasa este unic	Local per fragment
EVENIMENT_i	id_eveniment este unic	Local per fragment

BILET_i	id_bilet este unic	Local per fragment
PRODUS_i	id_produs este unic	Local per fragment
CLIENT (replica)	id_client este unic	Local per replica
CLIENT (replica)	email este unic	Local per replica
PLAN (replica)	denumire_plan este unic	Local per replica

ii. unicitate globală pe fragmente orizontale

Avand in vedere ca fragmentarea orizontala imparte datele pe 3 servere, cheile primare si celelalte campuri unice la nivel local trebuie sa respecte constrangerea de unicitate globala de asemenea.

Tabela	Constrangere de unicitate globala	Mecanism de aplicarea a unicitatii
ORAS	id_oras e unic pe toate ORAS_1, ORAS_2, ORAS_3	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
ADRESA	id_adresa e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
SALA_DE_FITNESS	id_sala e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
DEPARTAMENT	id_departament e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
ANGAJAT	id_angajat e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare

APARAT_FITNESS	id_aparat e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
CLASE_DE_FITNESS	id_clasa e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
EVENIMENT	id_eveniment e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
BILET	id_bilet e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
PRODUS_COMERCIAL	id_produs e unic pe toate fragmentele	Folosim chei primare care incep care incep cu ultima cheie primara deja existenta + i (unde i este id-ul serverului – 1,2 sau 3) si se incrementeaza cu 3 la inserare
ABONAMENT	id_abonament e unic pe serverul global	Unic global deoarece apare doar pe serverul global (fara replicare)

CLIENT	id_client e unic	Asigurat prin sincronizarea tabelelor de pe fiecare server
CLIENT	email e unic	Asigurat prin sincronizarea tabelelor de pe fiecare server

iii. unicitate globală în cazul în care trebuie să fie unică o combinație de coloane care se găsesc în fragmente verticale diferite, specificându-se argumentele care au stat la baza acestei decizii din punct de vedere al optimizării

Pentru fragmentele verticale ale tabeli ANGAJAT (ANGAJAT_V1 si ANGAJAT_V2), avem urmatoarele:

- Combinatia de attribute (id_departament , functie) din ANGAJAT_V2 nu trebuie sa fie unica, deoarece mai multi angajat pot avea aceeasi functie in acelasi departament.
- In cazul in care am avea nevoie ca o combinatie de attribute din cele doua fragmente verticale sa fie diferite (de exemplu: (email, functie)), ar trebui sa facem join la ANGAJAT_V1 si ANGAJAT_V2 pentru a verifica. Avand in vedere ca ambele fragmente verticale se afla pe acelasi server local, pentru fiecare regiune in parte, acest join este local si nu implica consturi aditionale de retea.

b. (0,5p) Cheie primară (la nivel local/global)

Nivel local

Fiecare fragment respecta constrangerea de cheie primara:

Fragment	Cheie primara
ORAS_i	id_oras
ADRESA_i	id_adresa

SALA_i	id_sala
DEPARTAMENT_i	id_departament
ANGAJAT_V1_i	id_angajat
ANGAJAT_V2_i	id_angajat
APARAT_i	id_aparat
CLASE_i	id_clasa
EVENIMENT_i	id_eveniment
BILET_i	id_bilet
PRODUS_i	id_produs
ABONAMENT_i	id_abonament
CLIENT	id_client
PLAN	denumire_plan

Nivel Global

Constrangerea de cheie primara global este asigurata de strategia de atribuire a cheilor primare descrie la exercitiul 8.a.ii. Vizualizarile globale vor prezenta cheile primare pentru toate fragmentele.

De exemplu, vizualizarea ANGAJAT_GLOBAL o sa aibe cheia sa primara virtual unica garantata si va putea contine toate tuplurile de pe toate serverele.

De asemea, toate cheile primare vor fi diferita de NULL, constrangere ce se poate aplica indiferent daca modelul este distribuit sau nu.

Pentru tabela CLIENT (care este replicat pe toate serverele) cheia primara id_client este identica pe toate replicile sale. La inserare, serverul global se va ocupa de generarea id-ului si distribuirea noului tuplu pe toate cele 3 servere locale.

c. (0,5p) Cheie externă (la nivel local și pentru relații stocate în baze de date diferite)

Nivel Local

Fragment	Cheie externa	Referinte
ADRESA_i	id_oras	ORAS_i
SALA_i	id_adresa	ADRESA_i
DEPARTAMENT_i	id_sala	SALA_i
ANGAJAT_V2_i	id_departament	DEPARTAMENT_i
APARAT_i	id_sala	SALA_i
CLASE_i	id_sala	SALA_i

EVENIMENT_i	id_sala	SALA_i
BILET_i	id_eveniment	EVENIMENT_i
BILET_i	id_client	CLIENT (replica locala)
PRODUS_i	id_sala	SALA_i
PRODUS_i	id_client	CLIENT (replica locala)
ABONAMENT_i	id_client	CLIENT (replica locala)
ABONAMENT_i	denumire_plan	PLAN (replica locala)

Toate cheile externe pot fi aplicate local pentru ca tabela parinte se afla pe acelasi server, astfel aceasta poate fi:

- Situata pe acelasi fragment din cauza fragmentarii efective, sau
- Tabela replicata pe toate cele 3 servere, deci disponibila local

Relatii pe servere diferite

In modelul nostru, nu avem relatii ce folosesc cheie straina intre doua servere diferite.

d. (0,5p) Validare (la nivel local și pentru relații stocate în baze de date diferite)

Nivel Local

Tabela	Constrangere	Tip
--------	--------------	-----

ORAS_i	regiune IN ('Transilvania', 'Muntenia', 'Moldova')	CHECK
SALA_i	Validarea formatului email-ului (LIKE '%@%.%')	CHECK
SALA_i	telefon IS NOT NULL	NOT NULL
DEPARTAMENT_i	salariu_minim > 0	CHECK
DEPARTAMENT_i	salariu_maxim > salariu_minim	CHECK
DEPARTAMENT_i	nume IS NOT NULL	NOT NULL
ANGAJAT_V1_i	Validarea formatului email-ului	CHECK
ANGAJAT_V1_i	nume IS NOT NULL AND prenume IS NOT NULL	NOT NULL
ANGAJAT_V2_i	salariu > 0	CHECK
ANGAJAT_V2_i	functie IS NOT NULL	NOT NULL
APARAT_i	data_productie <= SYSDATE	CHECK
EVENIMENT_i	data >= SYSDATE (pentru evenimente noi)	CHECK

EVENIMENT_i	numar_bilete > 0	CHECK
BILET_i	pret > 0	CHECK
PRODUS_i	pret > 0	CHECK
PRODUS_i	data_expirare > SYSDATE (pentru produse noi)	CHECK
ABONAMENT_i	data_sfarsit > data_start	CHECK
CLIENT	Validarea formatului email-ului	CHECK
CLIENT	email IS NOT NULL	NOT NULL
PLAN	pret > 0	CHECK

Relații stocate în baze de date diferite

Constrangere	Descriere	Aplicare
ANGAJAT - salariul sa fie in intervalul definit in tabela DEPARTAMENT	salariu BETWEEN salariu_minim AND salariu_maxim pentru departamentul din care face parte angajatul	Aplicat local printr-un trigger pe ANGAJAT_V2_i care face join cu DEPARTAMENT_i (amandoua apartinand aceluasi server)
BILET – numarul de bilete nu trece peste capacitatea evenimentului	COUNT(BILET where id_eveniment = X) <= numar_bilete pentru evenimentul X	Aplicat local printr-un trigger pe BILET_i, verificand EVENIMENT_i (acelasi server)
ABONAMENT – verificare a datii, sa nu se suprapuna	Un client nu trebuie sa aiba abonamente suprapuse	Un trigger care este apelat la inserare si verifica in tabela ABONAMENT daca un client are mai multe abonamente valide cu aceasi perioada

9. (0,25p) Formularea în limbaj natural a unei cereri SQL complexe care va folosi date din mai multe fragmente și va fi optimizată în etapa de implementare. Precizarea tehnicilor de optimizare ce ar putea fi utilizate pentru această cerere particulară (avantaje / dezavantaje de utilizare pentru o anumită tehnică)

Exercițiul 9

Formulare în limbaj natural

O interogare complexă potrivită pentru modelul nostru distribuit este:

„Pentru fiecare regiune (Transilvania, Muntenia, Moldova), să se determine sala de fitness care a obținut cele mai mari încasări din biletele vândute pentru evenimentele organizate în ultimele 90 de zile, afișând numele sălii, orașul, numărul de evenimente organizate, numărul total de bilete vândute, valoarea totală a încasărilor din bilete și numărul total de angajați ai sălii. Se iau în calcul doar sălile care au organizat cel puțin 2 evenimente și au cel puțin 10 angajați.”

Această interogare este complexă deoarece:

- folosește date din mai multe fragmente orizontale: `ORAS\_1..3`, `ADRESA\_1..3`, `SALA\_1..3`, `EVENIMENT\_1..3`, `BILET\_1..3`, `DEPARTAMENT\_1..3`
- folosește și fragmentarea verticală a relației `ANGAJAT`, dar doar fragmentul util pentru această interogare: `ANGAJAT\_V2\_1..3`
- conține filtre selective pe interval de timp
- conține agregări (`COUNT`, `SUM`)

- conține condiții de tip `HAVING`
- necesită un rezultat global, obținut prin reunirea rezultatelor locale de pe cele 3 servere

La nivel logic, interogarea trebuie implementată ca 3 subinterogări locale, câte una pe fiecare server regional, urmate de un `UNION ALL` și o selecție finală a sălii cu venitul maxim pe fiecare regiune.

O formă de implementare posibilă este:

Tehnici de optimizare potrivite pentru această interogare

1. Localizarea datelor și eliminarea fragmentelor nerelevante

Interogarea trebuie descompusă în 3 subinterogări locale, câte una pentru fiecare regiune, executate direct pe fragmentele locale (`\_1`, `\_2`, `\_3`). La final, serverul global reunește doar rezultatele agregate.

Avantaje:

- majoritatea join-urilor se execută local, pe același server
- se reduce traficul între servere
- se valorifică exact modul în care a fost făcută fragmentarea

Dezavantaje:

- implementarea este mai lungă decât o interogare scrisă direct peste tabele globale
- serverul global trebuie să coordoneze `UNION ALL` și agregarea finală

2. Predicate pushdown

Condiția pe timp (`e.data >= TRUNC(SYSDATE) - 90`) trebuie aplicată cât mai devreme, direct pe `EVENTIMENT\_i`, înainte de join-ul cu `BILET\_i`.

Avantaje:

- scade numărul de evenimente care intră în join
- scade numărul de bilete procesate
- reduce dimensiunea rezultatelor intermediare

Dezavantaje:

- beneficiul este mai mic dacă majoritatea evenimentelor sunt deja recente

3. Agregare locală timpurie

Încasările și numărul de bilete trebuie calculate întâi local, pe fiecare server, iar numărul de angajați trebuie calculat separat tot local. Abia apoi se face join-ul între rezultatele agregate și `SALA\_i`.

Avantaje:

- evită explozia cardinalității care ar apărea dacă am face direct join între `BILET\_i`, `EVENTIMENT\_i`, `DEPARTAMENT\_i` și `ANGAJAT\_V2\_i`
- serverul global primește puține rânduri, deja agregate
- planul de execuție devine mai stabil și mai ușor de analizat

Dezavantaje:

- interogarea devine mai complexă, deoarece necesită CTE-uri sau view-uri intermediare

4. Pruning pe fragmentul vertical al relației ANGAJAT

Pentru această interogare nu avem nevoie de nume, prenume, email sau telefon, deci nu trebuie reconstruită relația globală `ANGAJAT` din `ANGAJAT\_V1\_i` și `ANGAJAT\_V2\_i`. Este suficient `ANGAJAT\_V2\_i`, deoarece acesta conține `id\_departament`.

Avantaje:

- se citesc mai puține coloane
- se evită un join suplimentar între fragmentele verticale
- se reduce costul total al planului

Dezavantaje:

- tehnica este valabilă doar cât timp interogarea nu cere atribute din `ANGAJAT\_V1\_i`

5. Indexare pe atributele de filtrare și join

Pentru această interogare sunt utile indecși pe:

- `EVENTIMENT\_i(data, id\_sala)` sau separat pe `data` și `id\_sala`
- `BILET\_i(id\_eveniment)`
- `DEPARTAMENT\_i(id\_sala)`
- `ANGAJAT\_V2\_i(id\_departament)`
- `ADRESA\_i(id\_adresa, id\_oras)`

Avantaje:

- accelerează selecția evenimentelor recente
- accelerează join-urile dintre tabelele fragmentate
- reduc costul pentru planul ales de optimizer

Dezavantaje:

- ocupă spațiu suplimentar
- încetinesc operațiile de `INSERT`, `UPDATE` și `DELETE`

6. Execuție paralelă pe cele 3 servere regionale

Cele 3 subinterogări regionale pot fi lansate în paralel, iar serverul global așteaptă doar cele 3 rezultate agregate.

Avantaje:

- timpul total de răspuns se apropie de timpul celei mai lente subinterogări locale, nu de suma lor
- se pot folosi în paralel resursele tuturor serverelor

Dezavantaje:

- coordonarea execuției este mai complexă
- dacă un server regional este lent, el devine factorul limitativ

7. Materialized view / tabel sumar pentru rapoarte frecvente

Dacă această interogare devine una de raportare executată des, se poate construi un rezumat materializat cu încasările din bilete pe sală și pe perioadă, iar optimizer-ul poate rescrie interogarea către acel rezumat.

Avantaje:

- reduce mult costul pentru interogările repetate

- scade numărul de join-uri și agregări calculate la execuție

Dezavantaje:

- necesită spațiu suplimentar
- implică refresh și cost de întreținere
- există risc de date ușor întârziate dacă refresh-ul nu este imediat

Concluzie

Interogarea propusă este potrivită pentru exercițiul 9 deoarece:

- folosește date din mai multe fragmente orizontale
- valorifică și fragmentarea verticală
- poate fi optimizată clar la implementare prin localizare, pushdown de predicate, agregare timpurie, indexare și execuție paralelă
- produce un bun exemplu pentru etapa următoare, în care vor fi comparate planul rule-based și planul cost-based